Module 1:

Harness Engineering for Coding Agents

Instructions, state, scope, verification: build each layer, then measure it

{ik} | INTERVIEW KICKSTART

Introduction: Who is teaching today

Your instructor, in 30 seconds:

- Where they ship code with coding agents
- One agent failure they had to debug
- The harness layer they would add first
- Where to reach them after class

Welcome to today's class, before we begin, pop into the chat:

Name

Role



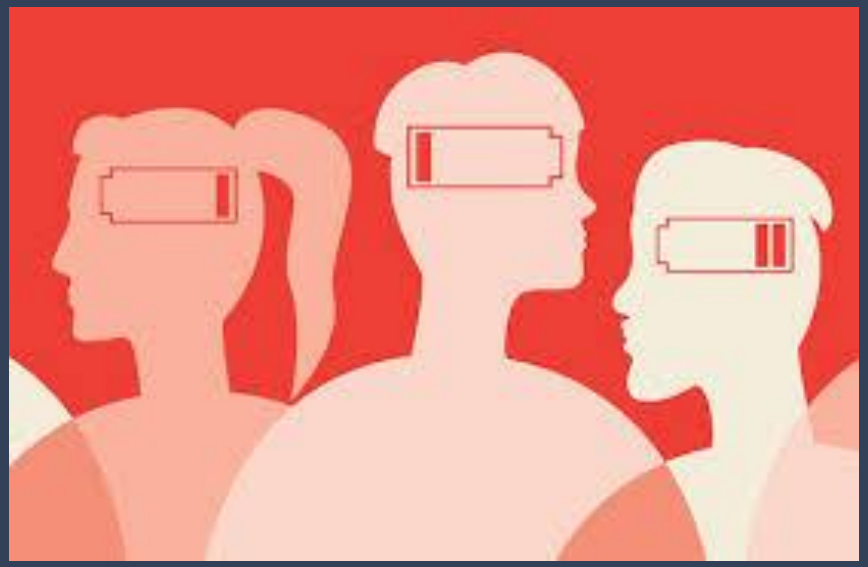
Location

The coding agent you use

{ik} | INTERVIEW KICKSTART

Structure of Class

1. **Live class -** 4 hr: four sections, two breaks, a final A/B
2. **Live demos -** every demo is a real model run
3. **Assignments -** four, one per section
4. **Review session -** 2 hr, one slot per assignment





{ik} | INTERVIEW KICKSTART

Optimize Your Experience

Ask, practise, and use your resources: what you put in is what you get out.

- **Uplevel** - recordings, resources, assignments
- **Wednesday technical coaching** - bring your queries
- **Discord** - peer learning
- **TAs** - answer questions in the live class
- **Support tickets** - raise one on Uplevel any time





{ik} | INTERVIEW KICKSTART

Success Hacks

- Be prepared for the class: finish the pre-class setup
- Attend every class
- Participate in class
- Attempt all the assignments
- Actively advocate for yourself
- Keep your goal in mind
- Consistency is the key

Be patient with yourself !

{ik} | INTERVIEW KICKSTART

Check Your Email for API Key

Use the OpenAI key from the program's email if you received one; otherwise create your own (Build tier) at platform.openai.com.

If you do not see this email:

- Please check your inbox
- Also check your spam or junk folder

If you still cannot find it, please reach out to the Operations Team operations@interviewkickstart.com

LangSmith key: a free account at smith.langchain.com

make doctor confirms the key can use the course models

.env.example

```
OPENAI_API_KEY=
LANGSMITH_API_KEY=
```

{ik} INTERVIEW KICKSTART

Clone & Practice: Your Learning Repository

harness-engineering-session · https://github.com/ANI-IN/harness-engineering-session

What's included

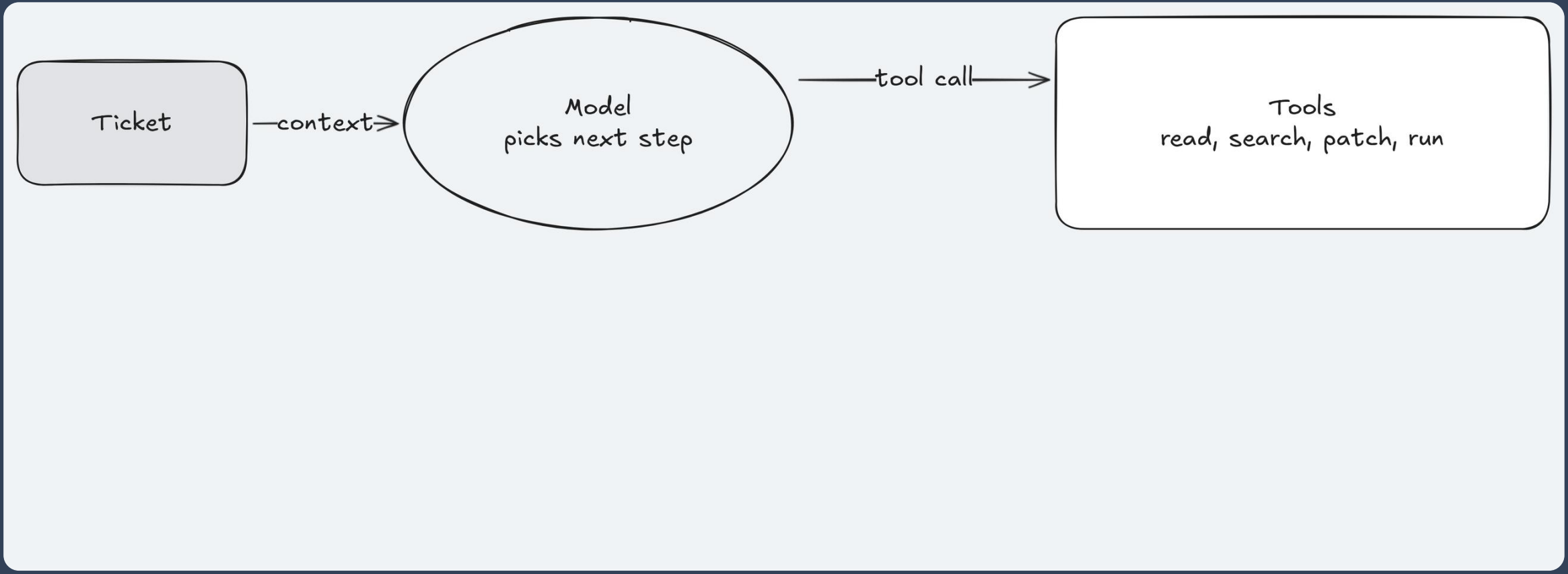
- Four section folders: concepts, demo, assignment
- Final project: one live A/B of every layer
- use-in-your-repo kit and four skills
- Slides and docs

Getting started

- `git clone https://github.com/ANI-IN/harness-engineering-session.git`
- Read `setup/README.md`
- `make setup`: creates `.env` from `.env.example`
- Fill in both keys, then `make doctor`

Recap: The Anatomy of an Agentic Loop

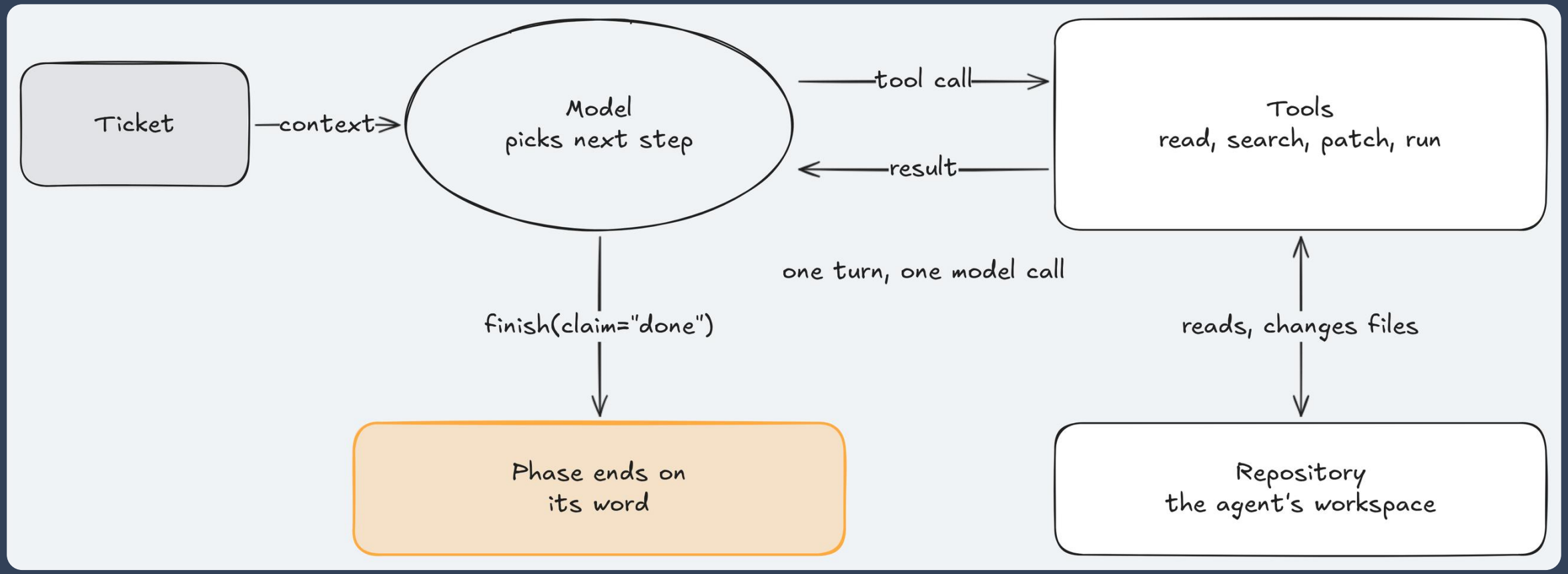
An agent is a loop: send the history to the model, run the tool it asks for, repeat.



Key idea: The model sees only what travels along these arrows.

Recap: The Anatomy of an Agentic Loop

The loop ends when the model stops calling tools, says done, or hits a limit.



Key idea: "Done" is the model's own claim. Nobody has checked it yet.

{ik} INTERVIEW KICKSTART

Today's Agenda

1. Instructions: what the agent knows
2. State and sessions: what it remembers
3. Scope and sandbox: what it may change
4. Verification: what proves it is done
5. Final A/B and all results

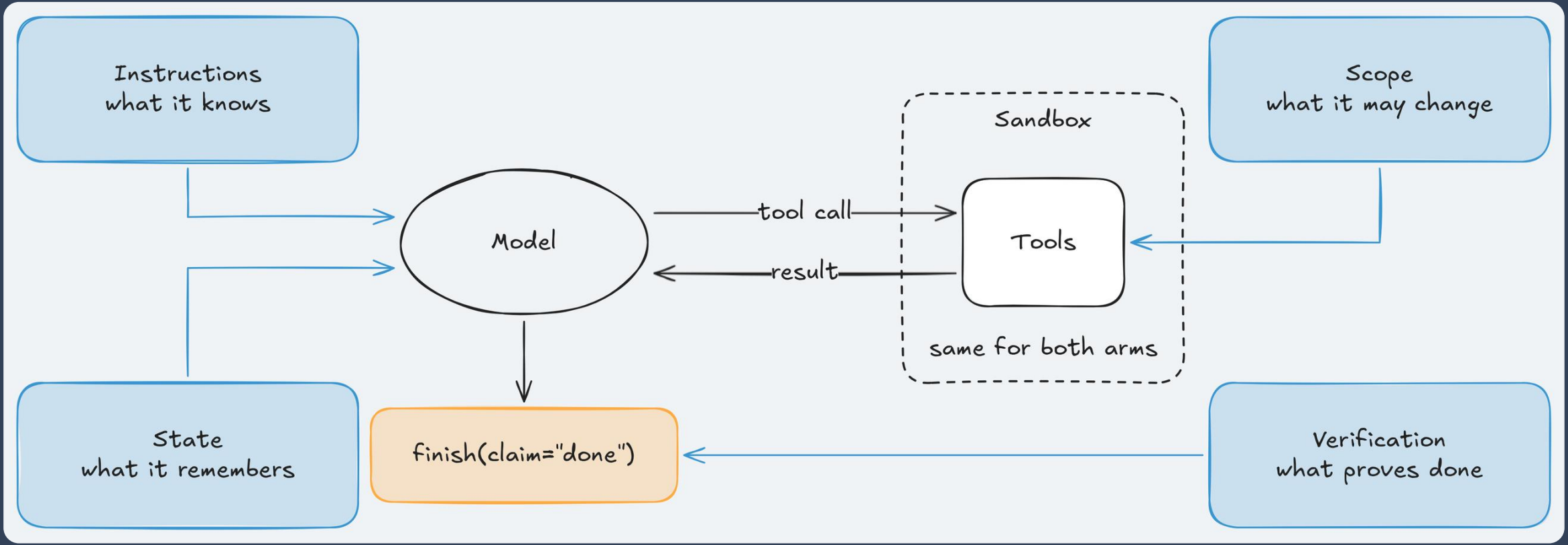
Break

Break

{ik} INTERVIEW KICKSTART

Problem statement: What are we building?

The harness around a coding agent, on TeamTrack, a small multi-tenant issue tracker.



Key idea: Same model; we change what it sees, remembers, may change, and must prove.

{ik} INTERVIEW KICKSTART

Problem statement: The problem in the real world

Your team gives a week of tickets on a multi-tenant issue tracker to a coding agent.

- **Monday:** copies an old handler with no tenant filter; tests pass, one customer reads another's issues
- **Tuesday:** a session stops mid-migration; the next writes a second migration for the same table
- **Wednesday:** a fix for one endpoint lands in shared middleware; an older feature breaks
- **Thursday:** "done" on an edit-conflict feature that still loses updates; the only evidence is its word

Each failure costs more than the ticket saved.

{ik} | INTERVIEW KICKSTART

The challenge is: The rules exist, the agent still misses

The rules, tests, and code are all in the repository. The agent still misses, because:

> *"anything it can't access in-context while running effectively doesn't exist."*
>
> — **Source 1: Harness engineering**

> *"each new session begins with no memory of what came before."*
>
> — **Source 2: Effective harnesses for long-running agents**

> *"Codex replicates patterns that already exist in the repository"*
>
> — **Source 1: Harness engineering**

> *"agents reliably skew positive when grading their own work"*
>
> — **Source 3: Harness design for long-running application development**

{ik} | INTERVIEW KICKSTART

Why this matters for Engineering Teams

Each failure costs the team; a harness costs too.

What a failure costs

- Tenant leak: incident, disclosure, pattern hunt
- Lost session: two migration heads, blocked deploy
- Scope creep: regression in an older feature
- False "done": review rests on trust

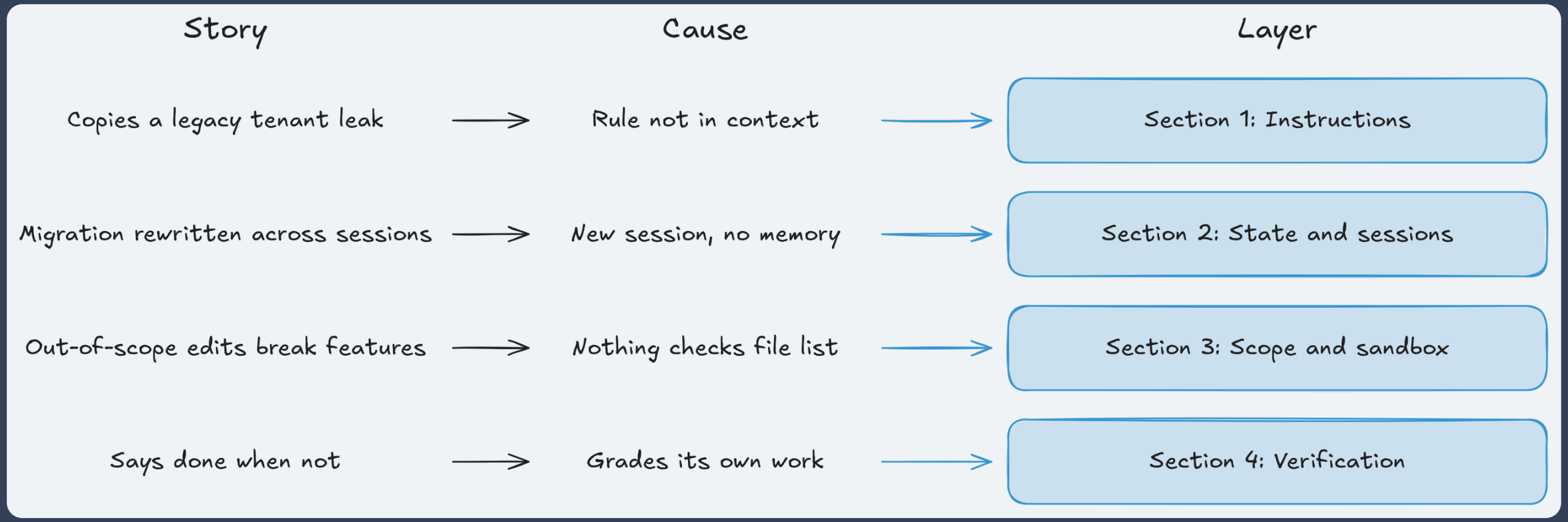
What a harness costs

- More turns per run
- Median 302 s against 190 s per arm
- Median $0.0123 against $0.0103 per arm

{ik} | INTERVIEW KICKSTART

Why We Need a Harness: The 4 Critical Gaps

Each story has a cause, and each cause gets one layer.

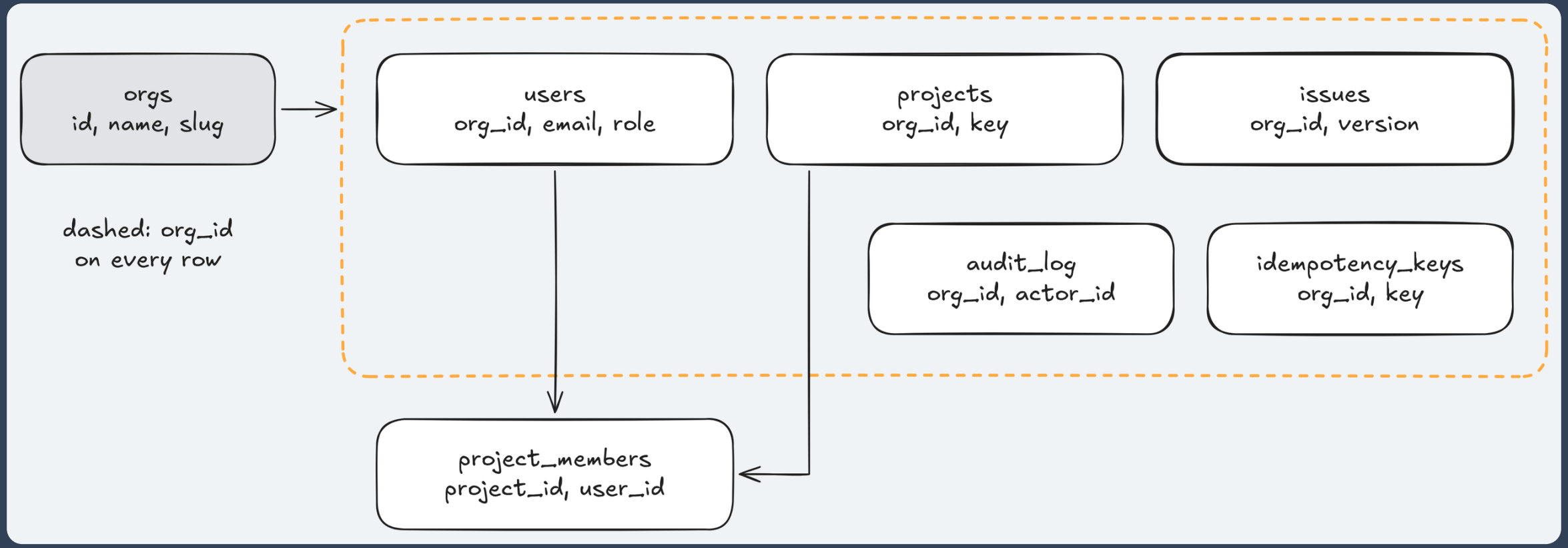


Key idea: One layer per cause: instructions, state, scope and sandbox, verification.

{ik} INTERVIEW KICKSTART

TeamTrack: The data model

Seven tables; two demo orgs, Acme and Globex.



Key idea: Every tenant-owned row carries org_id.

{ik} INTERVIEW KICKSTART

House rules: seven rules in plain words

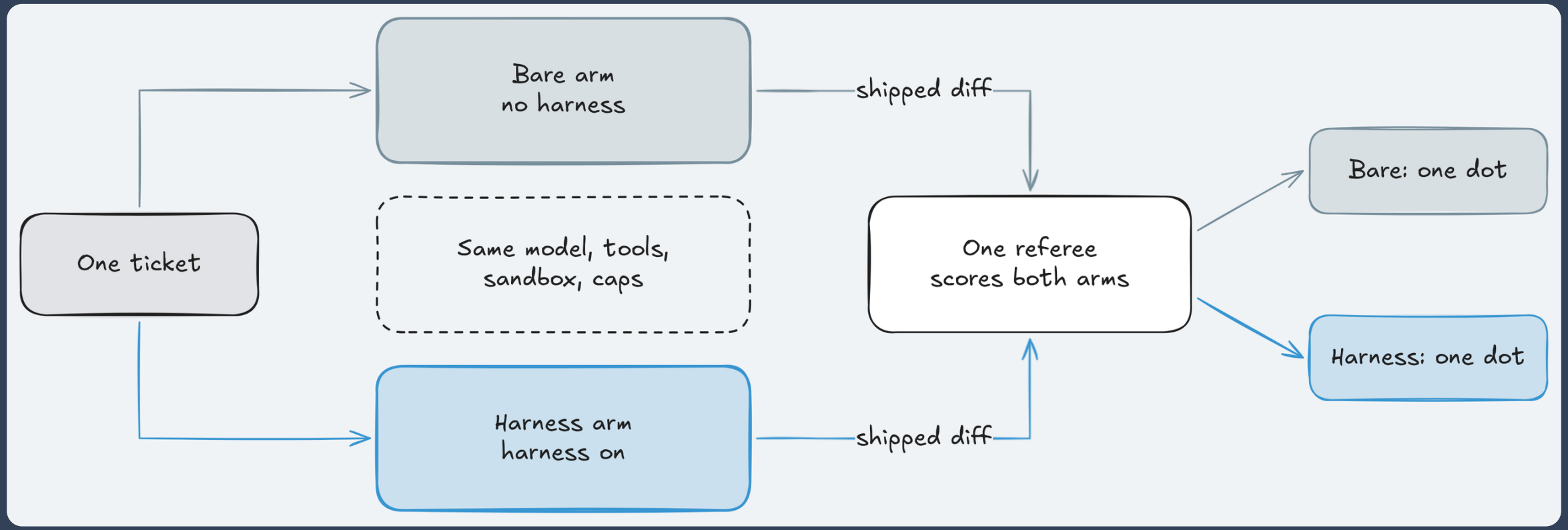
TeamTrack's handbook rules; both arms get all seven.

House rule	TeamTrack example
House rule 1: Tenant isolation	Another org's issue answers 404, never 403
House rule 2: Roles	A viewer's POST, PATCH or DELETE gets 403
House rule 3: API conventions	Lists return items and next_cursor
House rule 4: Safe retries	Same Idempotency-Key, one new project
House rule 5: Audit log	Every write adds exactly one audit row
House rule 6: Layering	app/api never imports app/db
House rule 7: Migrations	One Alembic head, fresh upgrade works

{ik} | INTERVIEW KICKSTART

How We'll Measure: Same ticket, two arms, one referee

Bare arm and harness arm get the same agent, model, ticket, tools, budgets, and sandbox.



Key idea: One referee, one primary measure chosen before any run, counts per arm.

Glossary: the terms used in every section

One line per term; come back here when a word is new.

Term	Meaning	Term	Meaning
Arm	One agent run, bare or harness	**Tenant**	One customer org and its data
Referee	Scores a run after the agent stops	**Idempotent**	A retry has no second effect
Primary measure	One question, chosen before any run	**Optimistic concurrency**	Stale save gets 412
Hidden tests	Never seen by the agent	**Alembic head**	Newest migration; two block upgrades
Mutant	Code with one planted fault	**Context reset**	Fresh session plus written handoff
Allowed paths	Files a phase may change	**Compaction**	Old context squeezed to a summary
Gate	Checks run when the agent says done	**Glob**	Path pattern; ** spans folders
Harness in action	Harness arm alone, live	**Least privilege**	Only the access the task needs
Held out	Hidden, used only to score	**Flaky test**	Passes and fails on the same code
Post hoc	Chosen after results; descriptive	**Ablation**	Remove one layer, measure again

Section 1: Instructions, what the agent knows

Today's Agenda



Launch, terminal B: `make show-1`

Failure story: The search that showed another org

An illustrative ticket: Feature 1: Org-wide issue search, GET /issues

1. Agent finds list_all_legacy next door
2. Copies it: short, already paginates
3. Tests create one org, so they pass
4. Review approves: looks like the file
5. After release: Acme sees Globex issues

{ik} INTERVIEW KICKSTART

Instructions: Why not paste every rule?

The tenant rule is in docs/HANDBOOK.md and both arms can open it. Can we paste every rule into the prompt?

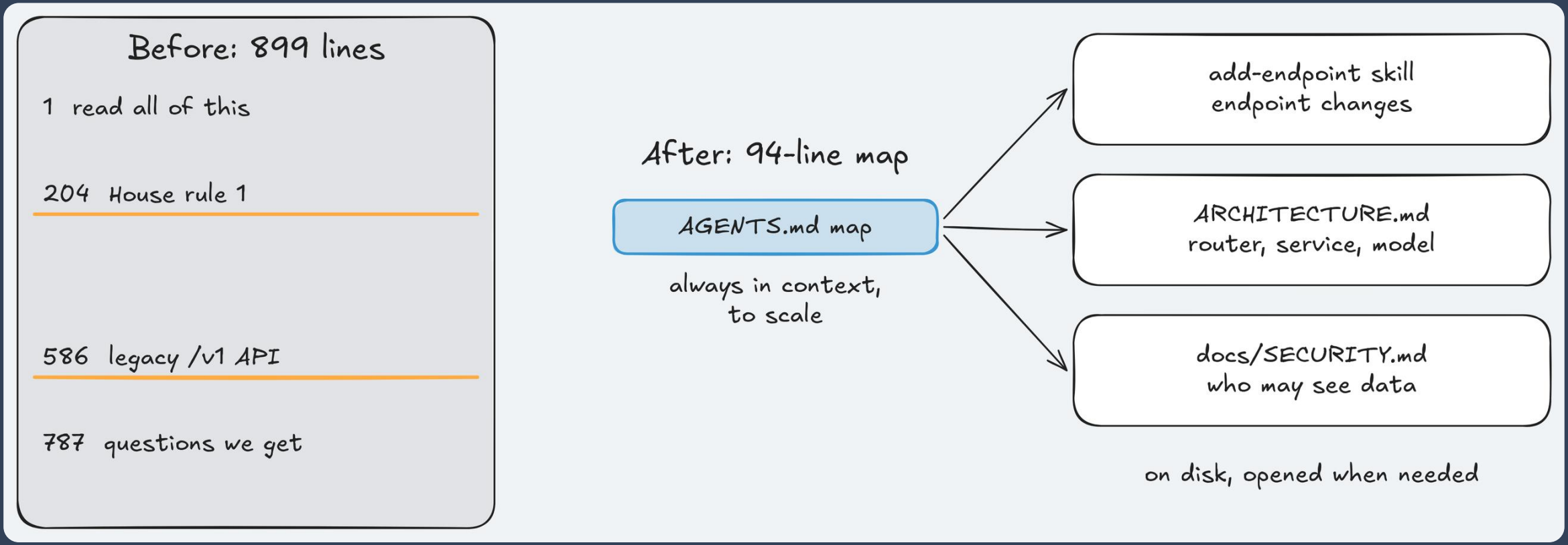
KEY IDEA

Not as the main mechanism: give the agent a map, not a manual.

1. It crowds out the task and the code
2. Too much guidance becomes non-guidance
3. It goes stale; no script checks it

1) A map, not a manual

AGENTS.md is a table of contents: about 100 lines, always in context.

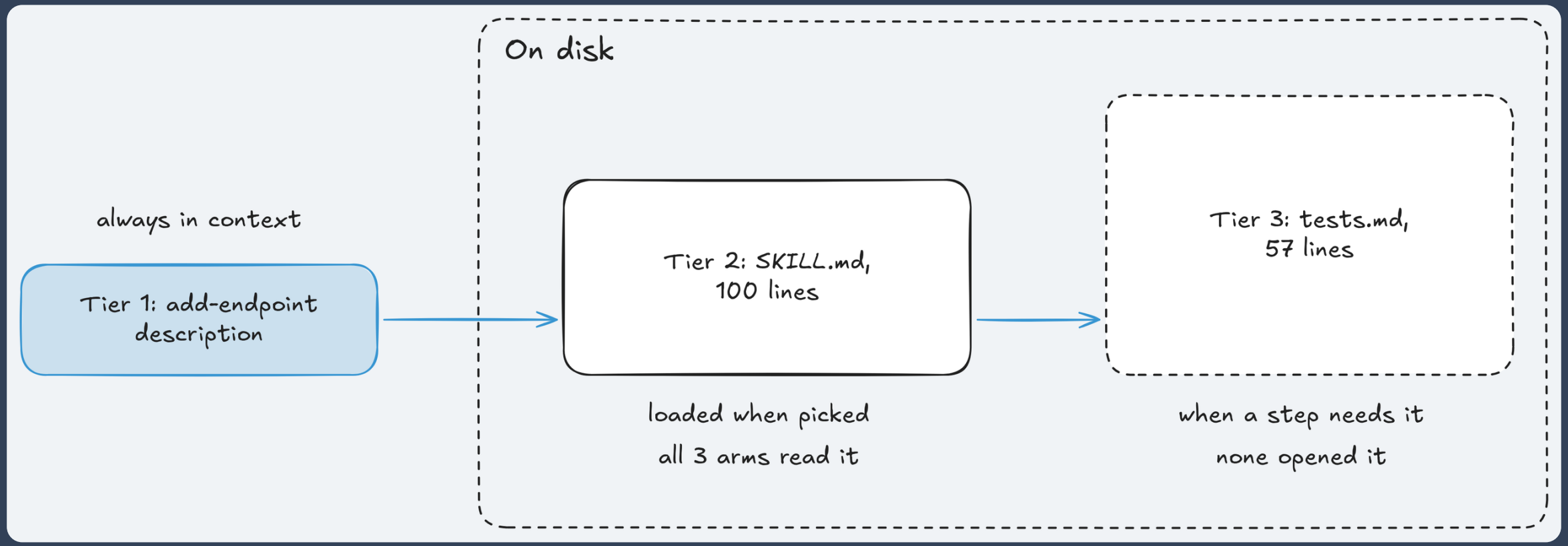


Key idea: The map says where each rule lives; it does not hold the rules.

{ik} INTERVIEW KICKSTART

2) Progressive disclosure: skills load in three tiers

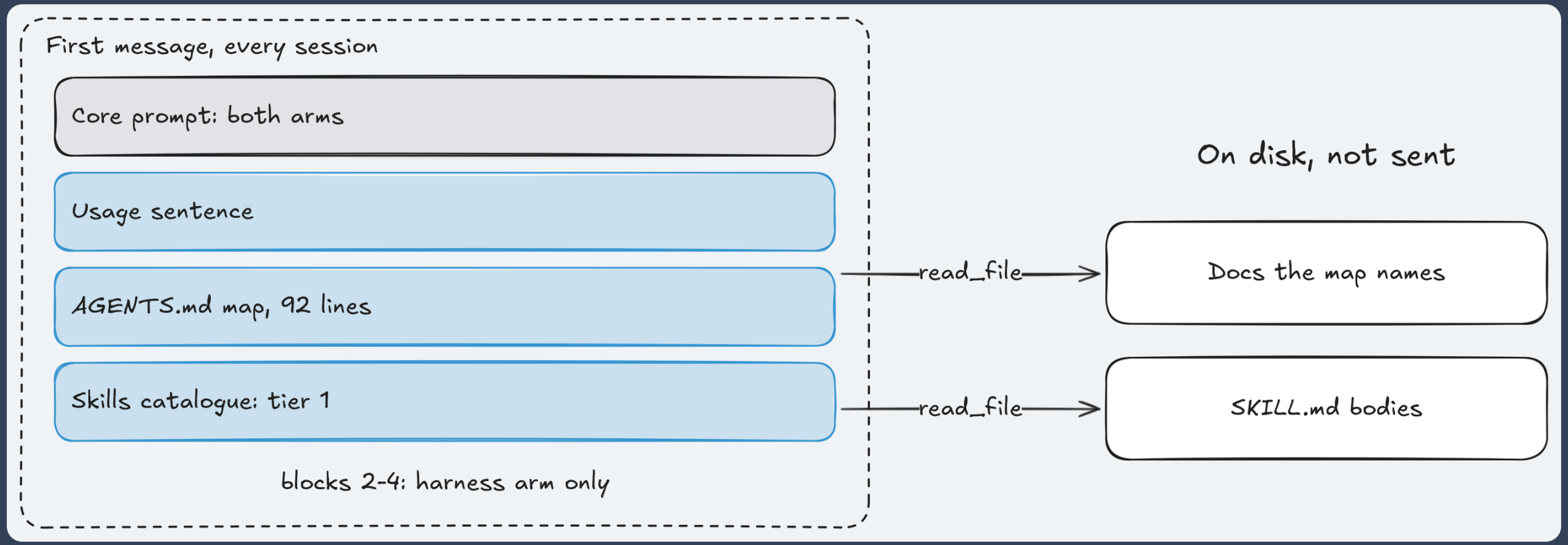
A small entry point first; the details only when the task needs them.



Key idea: Names and descriptions always; the body when picked; references when a step needs them.

3) The context prefix: small, fixed, out of reach

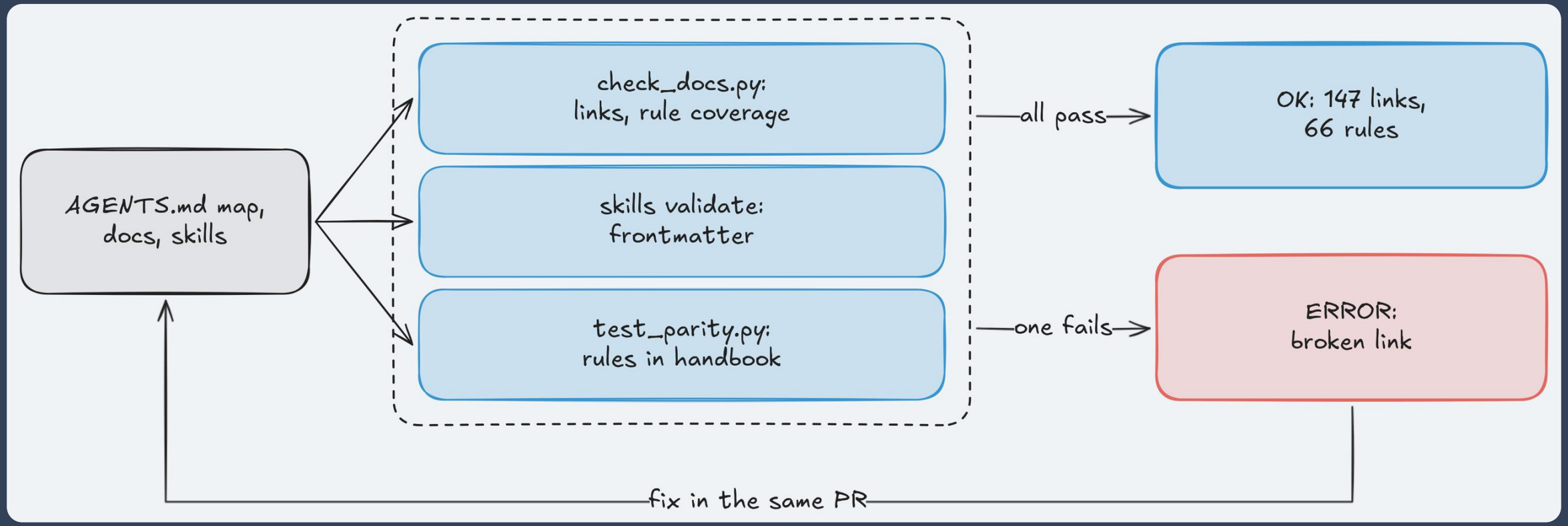
Sent first in every session: a usage note, the whole map, the skills catalogue.



Key idea: Built once from the course's copy, so agent edits to AGENTS.md change nothing.

4) Mechanical checks keep the map true

One big file can't be checked by a script. A map and focused docs can.



Key idea: Links resolve, each rule lives in one doc, retired rules stay out.

{ik} INTERVIEW KICKSTART

Cheat Sheet: The instructions layer in four moves

It informs. It never blocks.

Move	What it does
Map	About 100 lines, points to docs
Disclosure	Names first, bodies on demand
Prefix	Fixed, built from a read-only copy
Checks	Links, rule coverage, no new facts

{ik} | INTERVIEW KICKSTART

Section 1: Instructions - Implementation Notes: The map at work

DEMO - 1

{ik} | INTERVIEW KICKSTART

What we measured: No measurable difference on org-wide search

Earns its place when rules are many, far from code, and the code cannot show them.

What we measured



- A run fails if it adds a tenant leak
- Bare arm: 0 of 7 runs failed
- Harness arm: 0 of 7 runs failed

Apply it Monday

- ☐ Count your instruction file's lines
- ☐ One "read first" row per action
- ☐ "Do not copy" for legacy code
- ☐ A link and rule check in CI

No Question Is Too Small Every Question Deserves an Answer!!

Ask about anything:

- Concepts from today's sections
- How this applies to your work
- Implementation details
- Clarifications on what we covered

Quick Request

- Please do not mind if a question gets skipped accidentally
- Repost politely in chat if needed
- We will try our best to cover as many questions as possible

Remember: If you're wondering about it, others probably are too.



Time for a Break



A 10-minute break is starting now. Please return when the timer ends to continue the presentation

{ik} | INTERVIEW KICKSTART

Section 2: State and sessions, what it remembers



Launch, terminal B: `make show-2 SESSION_BUDGET=12,38`

{ik} INTERVIEW KICKSTART

Failure story: The second session redoes the first

An illustrative ticket: Feature 2: Comments on issues, over two sessions

1. Session 1 writes the comments migration, stops at its limit
2. Half an endpoint, uncommitted, no note left
3. Session 2 starts with an empty context
4. Writes a second migration from the old head
5. Two Alembic heads block the deploy

{ik} | INTERVIEW KICKSTART

Fresh session: Why redo work already on disk?

Session 2 has the whole repository but an empty context. Can we carry the old context over?

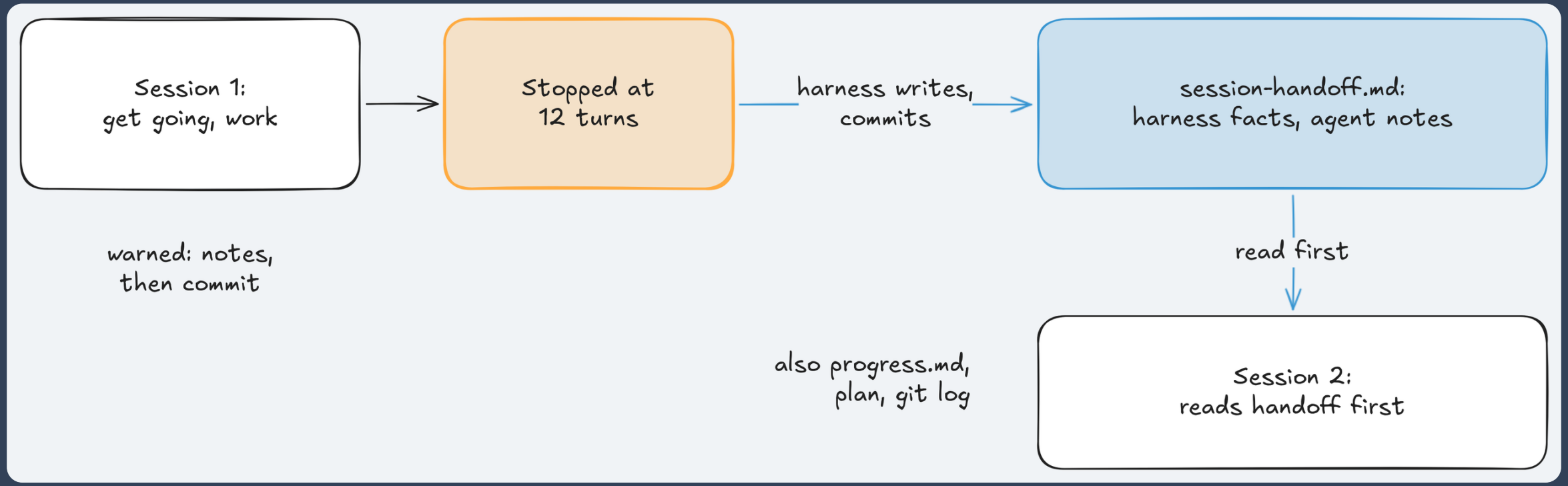
KEY IDEA

Not on its own: "compaction isn't sufficient." Reset, then hand over written state.

1. Compaction keeps a squeezed copy with gaps
2. The repository holds the work, but unread

1) Session lifecycle: a planned stop and reset

Every session: get up to speed, work and commit, wrap up, stop.



Key idea: At the stop, the harness writes the facts, whatever the agent did.

{ik} INTERVIEW KICKSTART

2) The handoff: two owners, one file

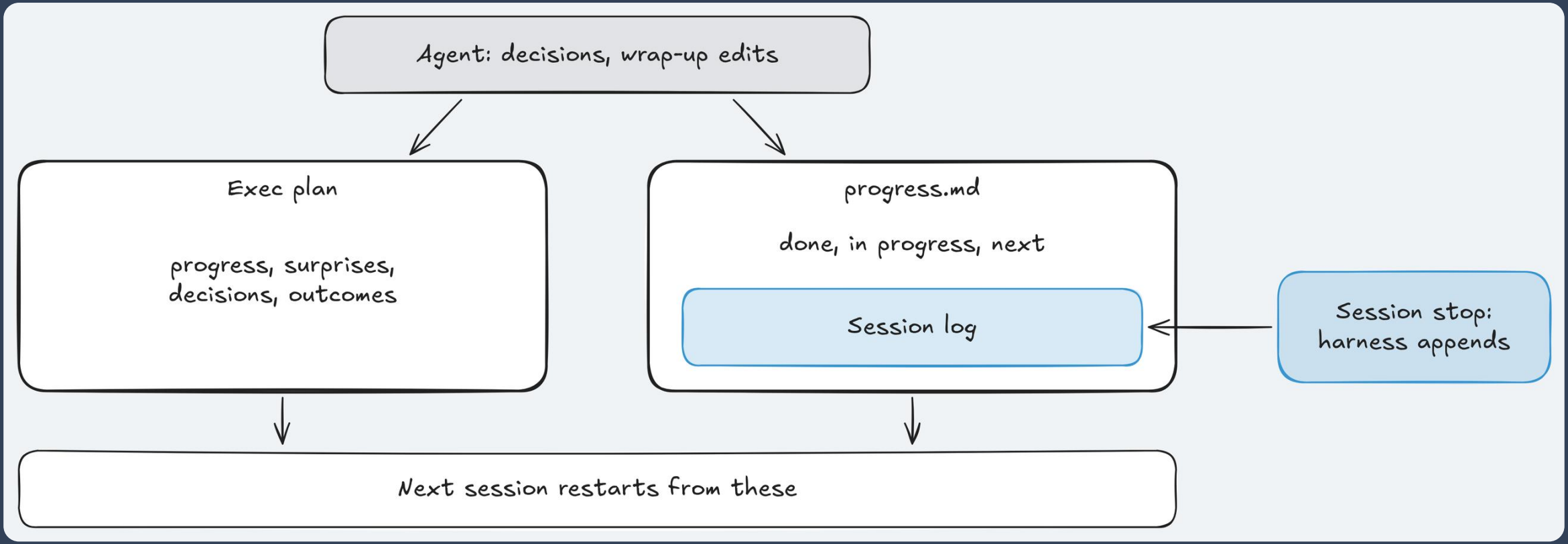
session-handoff.md carries the last session's state and the next steps.



Key idea: The harness writes the facts from git; the agent writes its notes; neither overwrites the other.

3) The exec plan and the progress ledger

Two checked-in files: progress.md (done, in progress, next) and the exec plan.

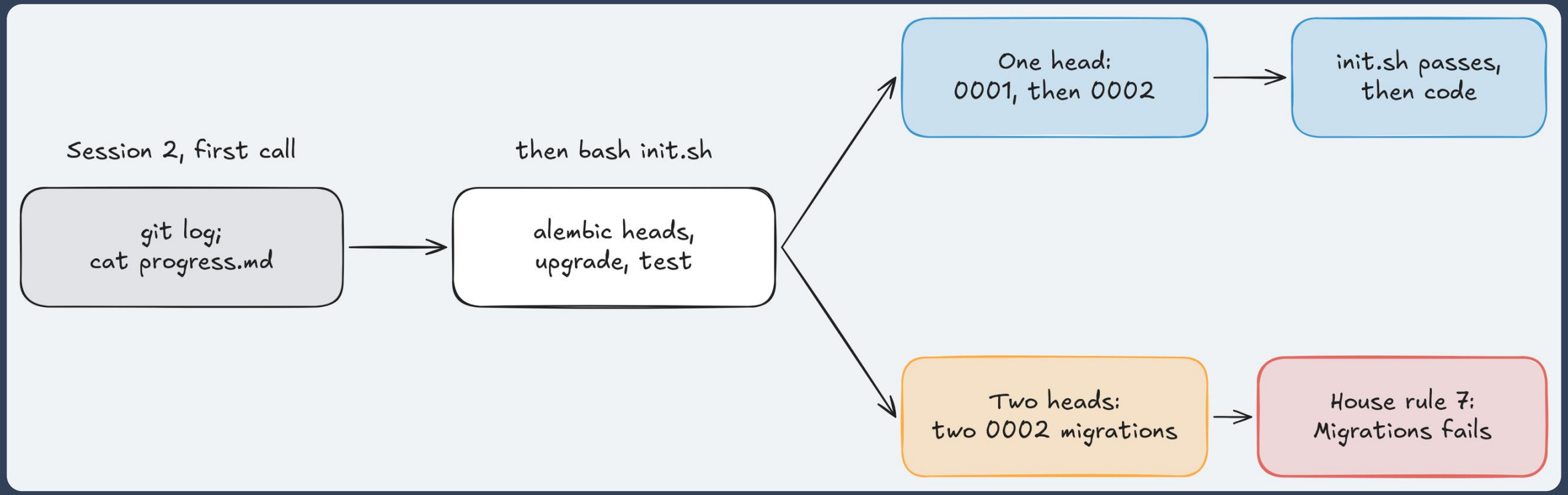


Key idea: "it should always be possible to restart from *only* the ExecPlan and no other work."

{ik} | INTERVIEW KICKSTART

4) Getting up to speed: init.sh before code

"Read the git logs and progress files to get up to speed on what was recently worked on."



Key idea: Read state, then run init.sh: one migration head and a basic test, before any code.

Cheat Sheet: Four state pieces across a stop

Four files, one job each; refer back when you add state to your repo.

File	Job	Who writes it
init.sh	Setup and a basic test	Harness
progress.md	What each session did, what's next	Both
Exec plan	Plan, decisions, restart bar	Agent
session-handoff.md	Harness facts plus agent notes	Two owners

{ik} | INTERVIEW KICKSTART

Section 2: State and sessions - Implementation Notes: Stop and resume

DEMO - 2

{ik} | INTERVIEW KICKSTART

What we measured: No measurable difference on comments

Earns its place when tasks outlast one session and half-done work is hard to see.

What we measured



- Continuity failures before class
- Bare arm: 0 of 7 runs
- Harness arm: 0 of 7 runs
- 6 of 12 arms built nothing

Apply it Monday

- ☐ Stop your agent mid-task, restart fresh
- ☐ Watch its first five tool calls
- ☐ Write facts from git at every stop
- ☐ Measure redone work; drop files that change nothing

No Question Is Too Small Every Question Deserves an Answer!!

Ask about anything:

- Concepts from today's sections
- How this applies to your work
- Implementation details
- Clarifications on what we covered

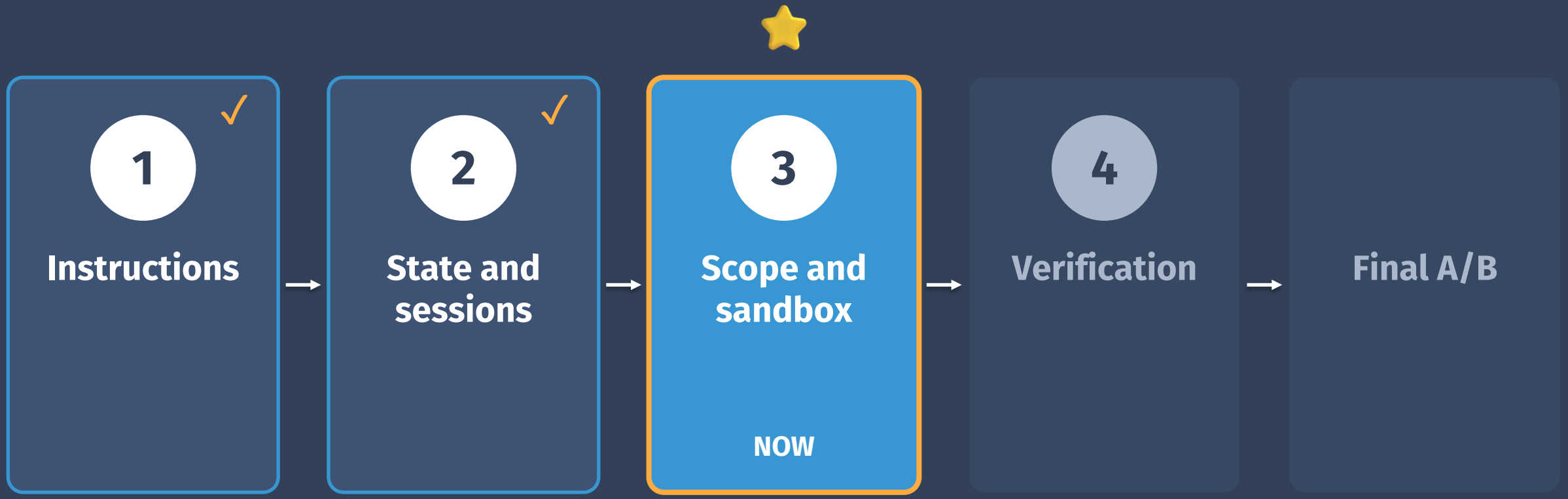
Quick Request

- Please do not mind if a question gets skipped accidentally
- Repost politely in chat if needed
- We will try our best to cover as many questions as possible

Remember: If you're wondering about it, others probably are too.



Section 3: Scope and sandbox, what it may change



Launch, terminal B: `make demo-3 FEATURE=version-checks`

Failure story: A tidy diff in a shared file

An illustrative ticket: Feature 4: Version checks on issue edits, with a list of files to change

1. Needs a new error class
2. Handbook points to app/services/errors.py
3. Edits a module every endpoint imports
4. Small diff, tests pass, approved
5. A month later: the wrong status, nobody links it

{ik} INTERVIEW KICKSTART

Scope: Is the ticket's file list enough?

The ticket lists the files to change, and both arms read the same list. Is that enough?

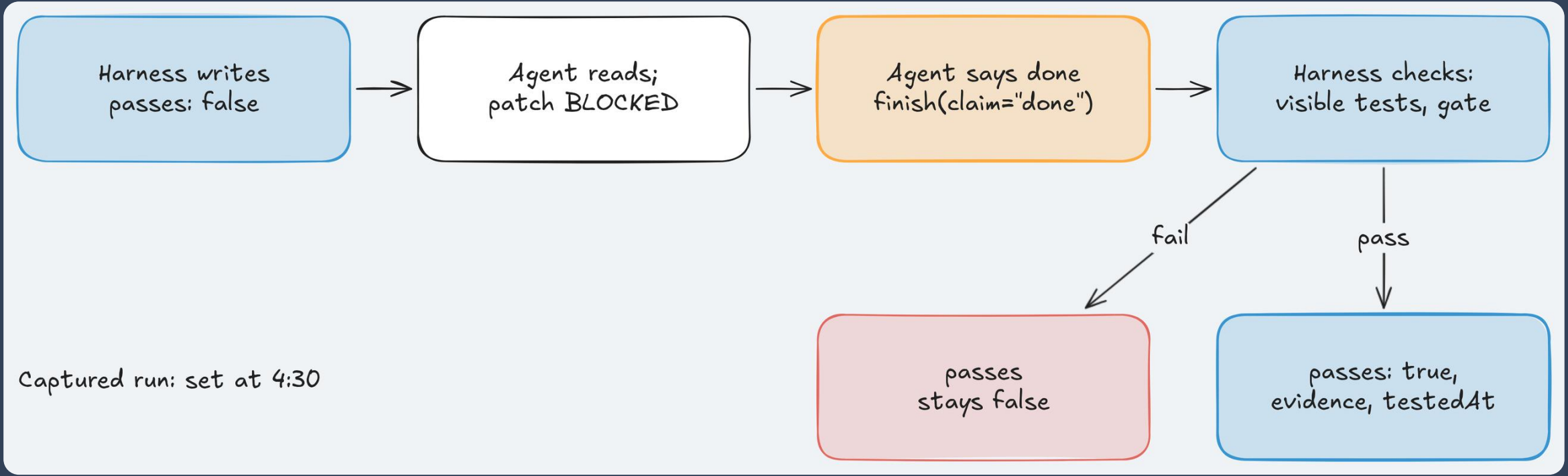
KEY IDEA

No: text is a request. Check every tool call in code.

1. Nothing checks the list; the handbook points elsewhere
2. "Less likely" is not "never"

1) One feature at a time: feature_list.json

The harness writes one feature with passes: false.

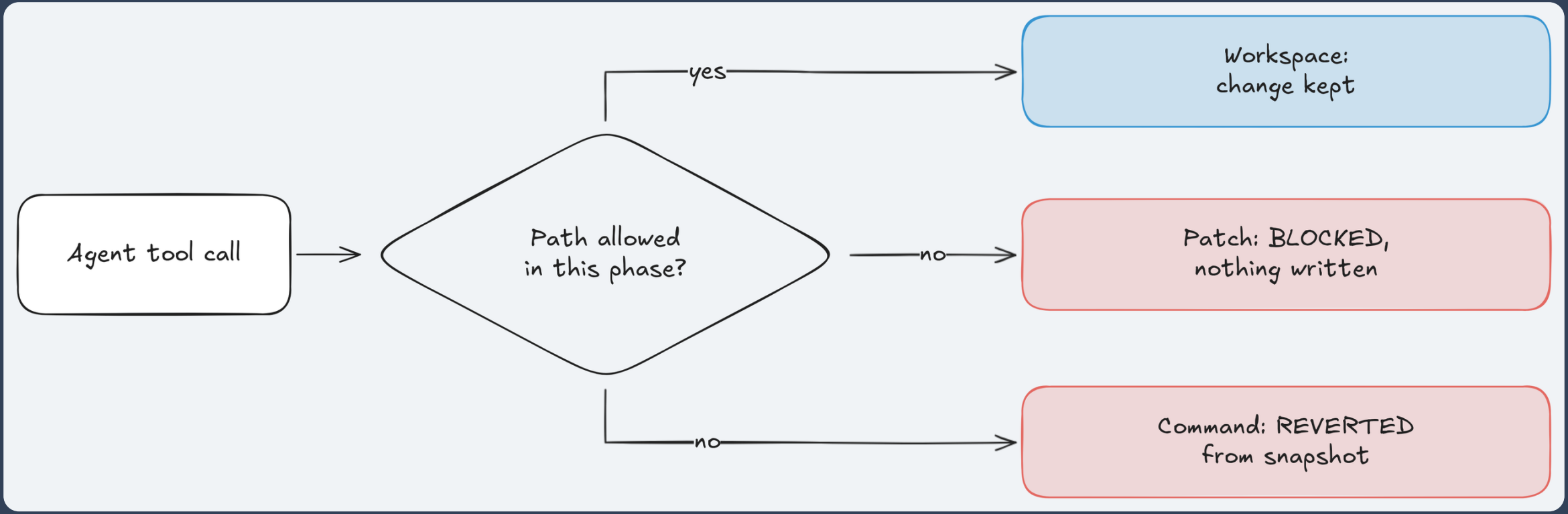


Key idea: Only the harness sets passes to true, from its own test run.

{ik} | INTERVIEW KICKSTART

2) Allowed paths per phase, checked by code

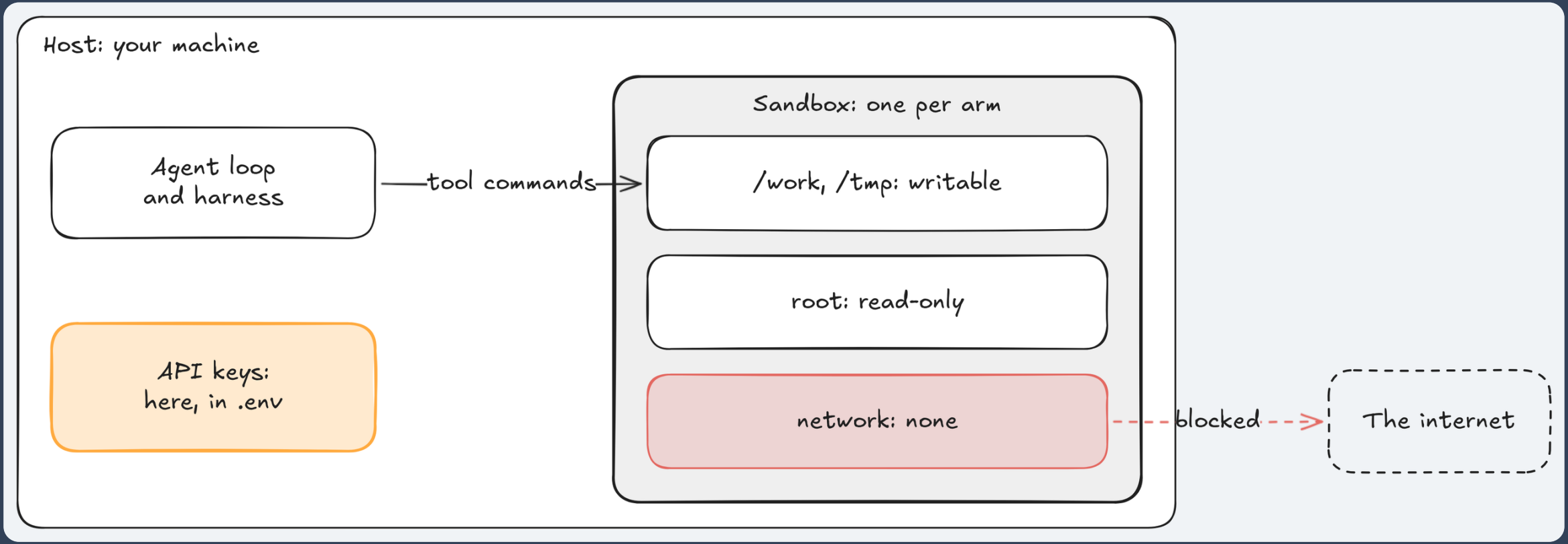
Every tool call is checked against the phase's allowed paths.



Key idea: A patch outside the list is BLOCKED; a command's stray writes are REVERTED.

3) The sandbox boundary: where the keys live

Every tool command runs in one container per arm; the loop and the keys stay on the host.

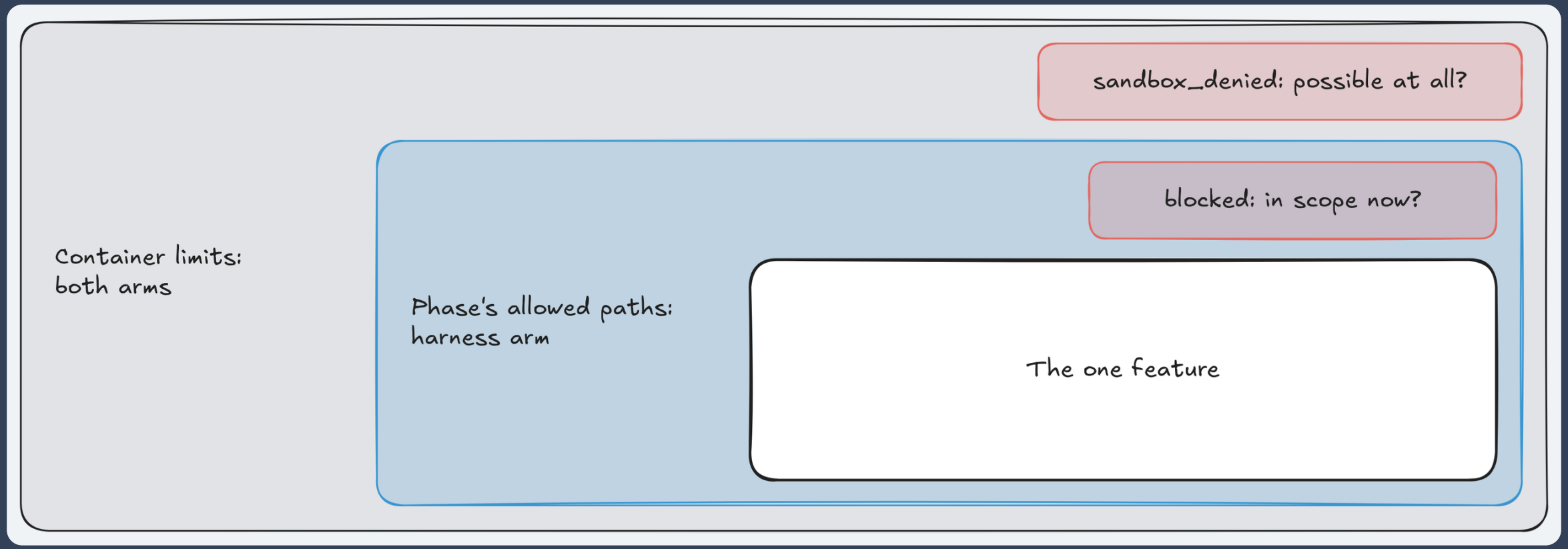


Key idea: No network, read-only root, the same container for both arms: never the A/B variable.

{ik} INTERVIEW KICKSTART

4) Least privilege: two refusals, two questions

"enforce boundaries centrally, allow autonomy locally."



Key idea: Scope asks "in scope now?"; the sandbox asks "possible at all?"

{ik} | INTERVIEW KICKSTART

Cheat Sheet: Four boundaries for one feature

Refer back to this when you scope your next agent task.

Boundary	What it does	Arm
feature_list.json	One feature; harness sets passes	Harness only
Allowed paths	Per phase; patch blocked, command reverted	Harness only
Sandbox	No network, keys on the host	Both
Least privilege	Non-root, caps, two refusal types	Both

{ik} | INTERVIEW KICKSTART

Section 3: Scope and sandbox - Implementation Notes: Scope at work

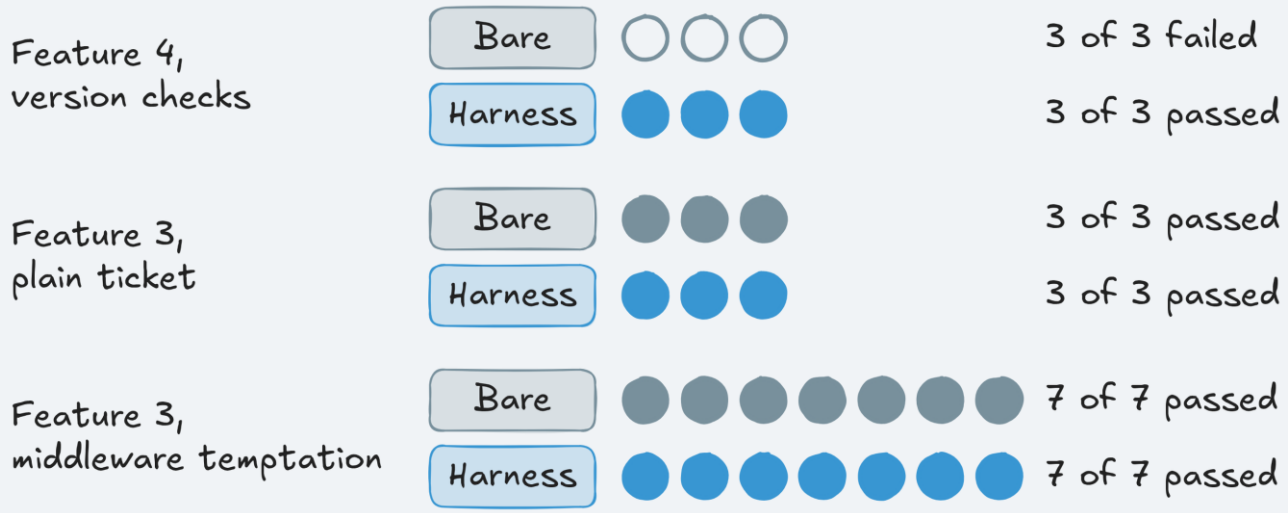
DEMO - 3

{ik} | INTERVIEW KICKSTART

What we measured: Scope held on version checks

Earns its place when a shared file looks like the natural home and you need a guarantee.

What we measured



- Runs with a file outside the allowed paths
- Version checks: bare 3 of 3, harness 0 of 3
- Safe retries: bare 0 of 10, harness 0 of 10
- Partly by construction

Apply it Monday

- ☐ Write the allowed paths as globs first
- ☐ Count `git diff --name-only` outside them
- ☐ Check patches before, commands after
- ☐ Keep grading files out of reach

No Question Is Too Small Every Question Deserves an Answer!!

Ask about anything:

- Concepts from today's sections
- How this applies to your work
- Implementation details
- Clarifications on what we covered

Quick Request

- Please do not mind if a question gets skipped accidentally
- Repost politely in chat if needed
- We will try our best to cover as many questions as possible

Remember: If you're wondering about it, others probably are too.



Time for a Break



A 5-minute break is starting now. Please return when the timer ends to continue the presentation

{ik} | INTERVIEW KICKSTART

Live Zoom Poll



We are launching a poll on Zoom. Please take 2 minutes to fill out the form.

Which feature runs as the final live A/B:

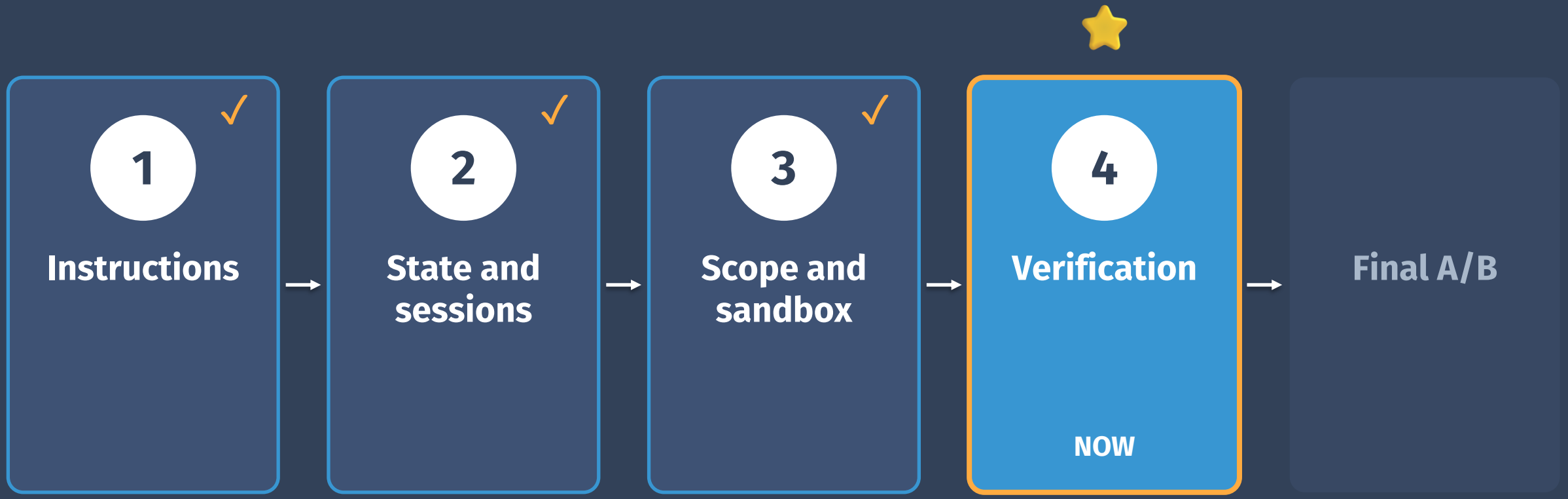
Feature 5: Bulk close
many issues in one request

Feature 6: Watchers
work spans a session stop

Feature 7: Labels
tag and filter issues

{ik} | INTERVIEW KICKSTART

Section 4: Verification, what proves the work is done



Launch: terminal B `make show-4`; terminal C `make final FEATURES=watchers,bulk-close`

Failure story: Done, but two edits still collide

An illustrative ticket: Feature 4: Version checks on issue edits, stop lost updates

1. Adds the header check to three endpoints
2. Writes a happy-path test
3. Runs the suite: all green
4. Reports "Done, the full suite passes"
5. After release: two editors still overwrite each other

{ik} INTERVIEW KICKSTART

Verification: Suite green. Are version checks done?

A bare arm before class: 191 visible tests passing, "Verified the complete visible suite", finish(claim="done"). Done?

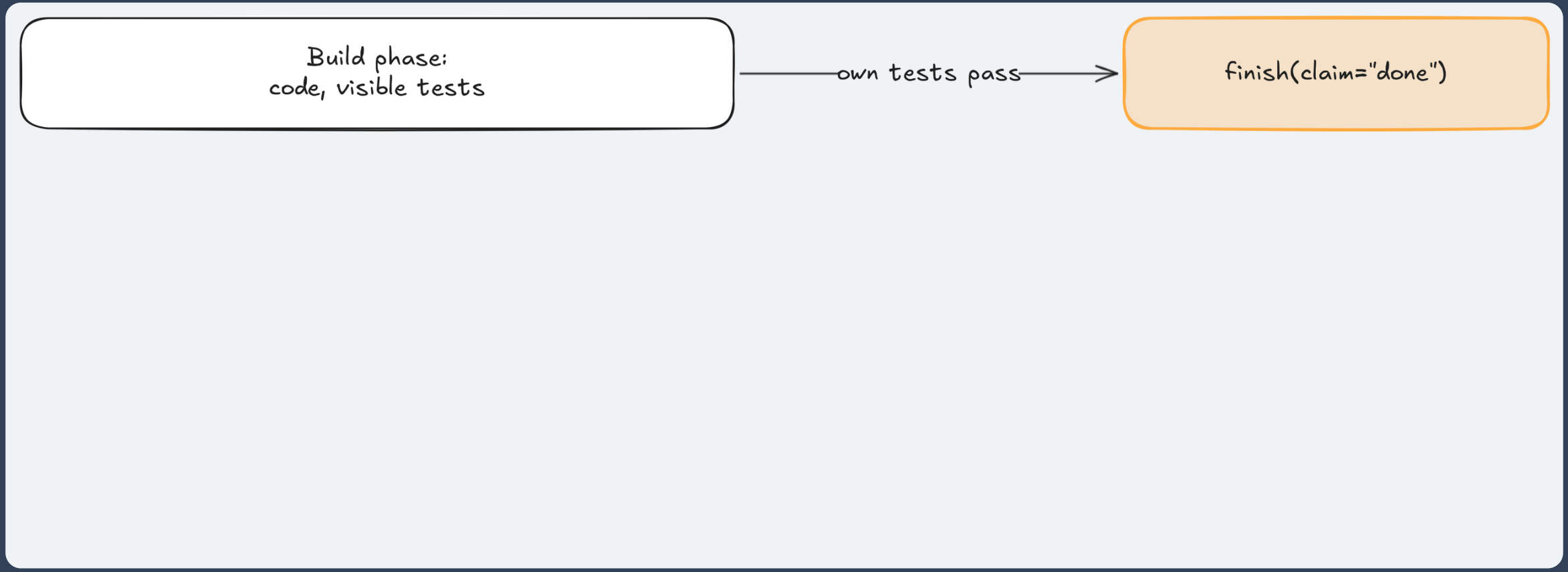
KEY IDEA

No: the referee failed it, 68 of 69 hidden tests. Who should decide "done"?

1. The visible suite missed a combined case
2. The agent graded its own work

1) Self-grading and visible tests

The agent writes the tests, runs them, and grades the result.

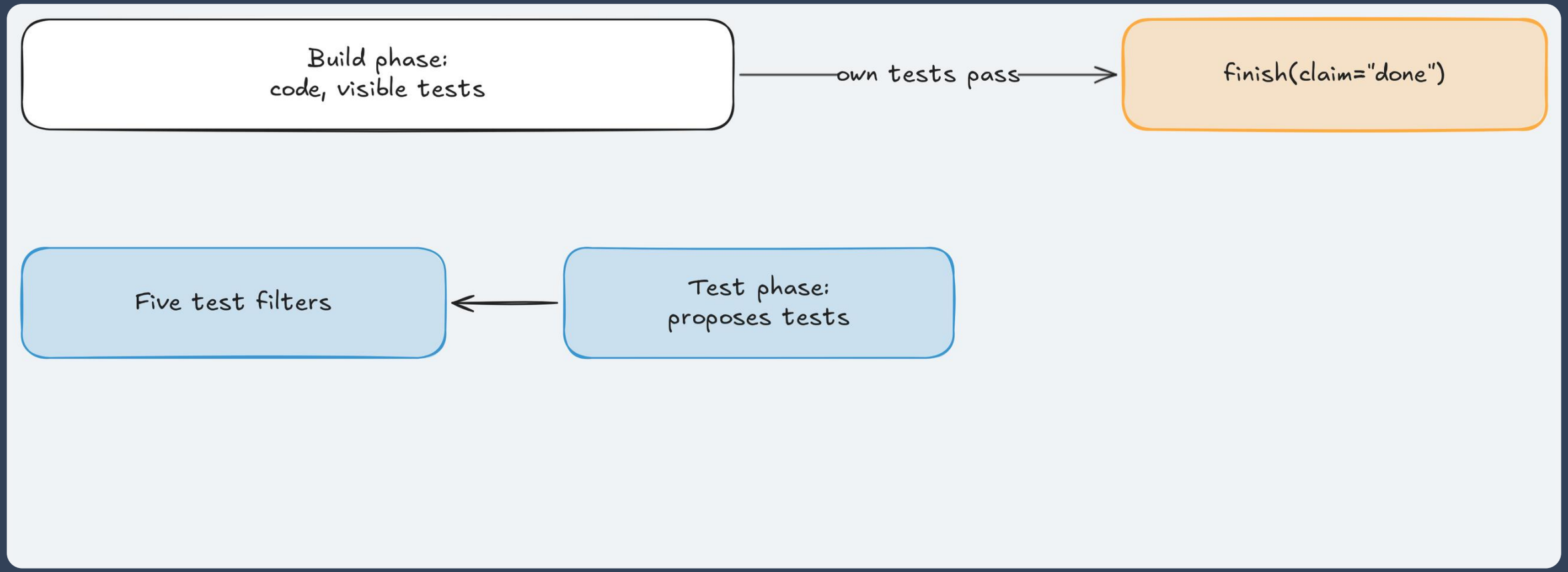


Key idea: Every agent saturates the visible suite; held-out tests still find failures.

{ik} INTERVIEW KICKSTART

2) Test-phase filters: keep tests that prove something

After the build, a short test phase; each proposed test passes five filters, cheapest first.

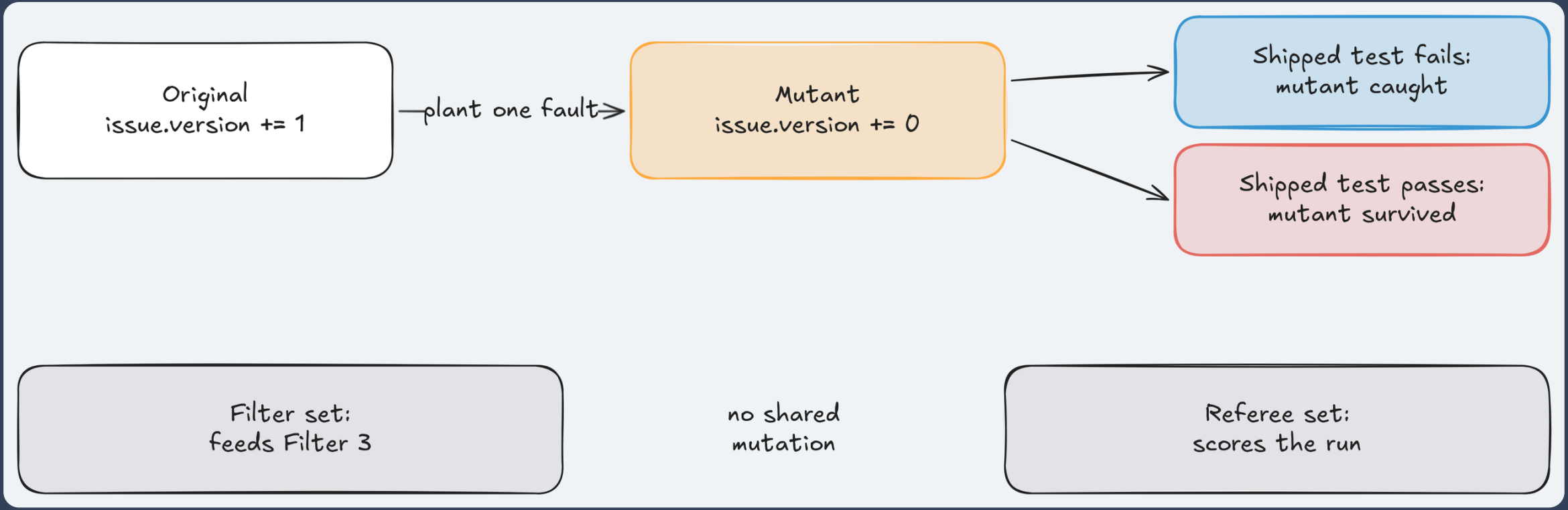


Key idea: Keep a test only if it runs, is stable, and adds something.

{ik} | INTERVIEW KICKSTART

3) Mutation testing and hidden tests

A mutant is a copy of the code with one small planted fault; a test that fails on it catches it.

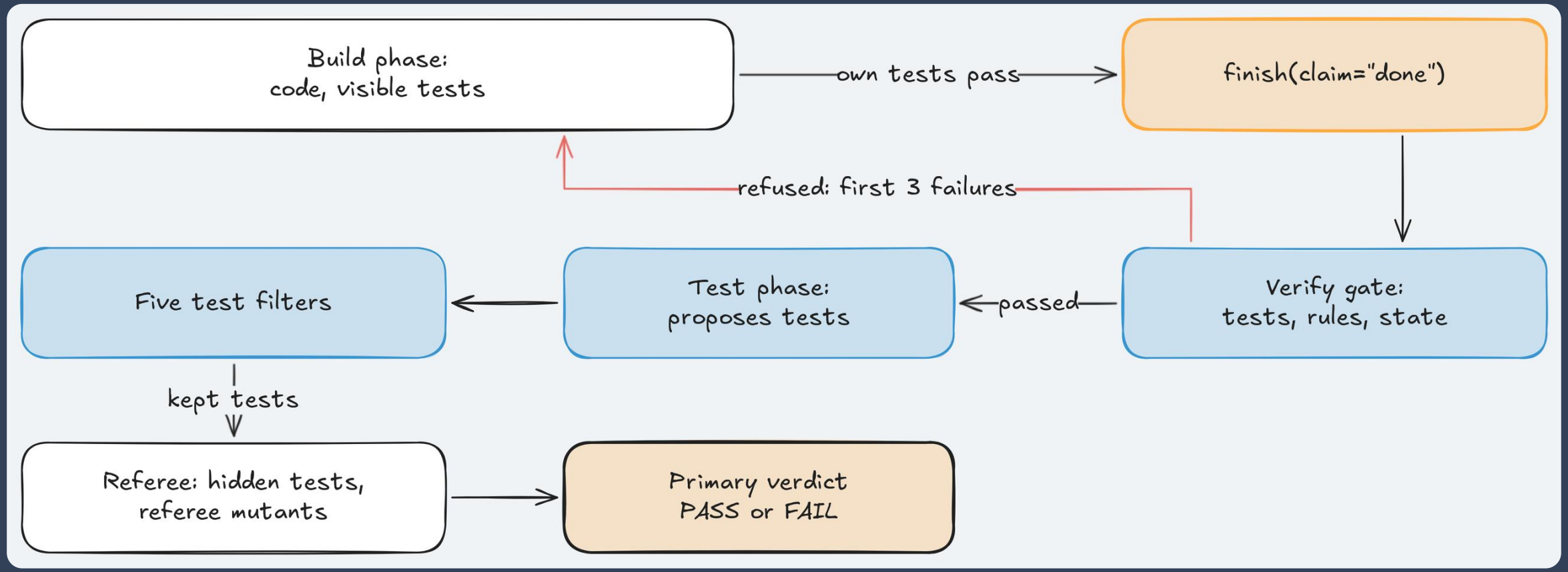


Key idea: Two held-out checks, owned by the referee: hidden tests and referee mutants.

{ik} | INTERVIEW KICKSTART

4) The gate on "done" and the separate referee

finish(claim="done") starts a check, not the end of the phase.



Key idea: Any gate failure returns "Not accepted"; the referee scores later, in its own container.

{ik} INTERVIEW KICKSTART

Cheat Sheet: four checks the agent does not own

Two give feedback, two stay held out.

Check	When	Asks
Verify gate	At the done claim	Tests, rules, clean state pass?
Test filters	After the test phase	Runs, stable, adds something?
Hidden tests	After the agent stops	Works in combination?
Referee mutants	After the agent stops	Would the tests notice?

Section 4: Verification - Implementation Notes: The gate at work

DEMO - 4

{ik} | INTERVIEW KICKSTART

What we measured: No measurable difference on version checks

Earns its place when the agent can claim done early and checks are cheap, exact, partly held out.

What we measured



- Pass: all 69 hidden tests, all 4 mutants
- Bare arm: failed 7 of 7 runs
- Harness arm: failed 7 of 7 runs
- No harness arm claimed done

Apply it Monday

- ☐ Run your checks when the agent says done
- ☐ Return three failures at most
- ☐ Run them where the agent cannot edit
- ☐ Plant one fault on your riskiest line

No Question Is Too Small Every Question Deserves an Answer!!

Ask about anything:

- Concepts from today's modules
- How this applies to your work
- Implementation details
- Clarifications on what we covered

Quick Request

- Please do not mind if a question gets skipped accidentally
- Repost politely in chat if needed
- We will try our best to cover as many questions as possible

Remember: If you're wondering about it, others probably are too.



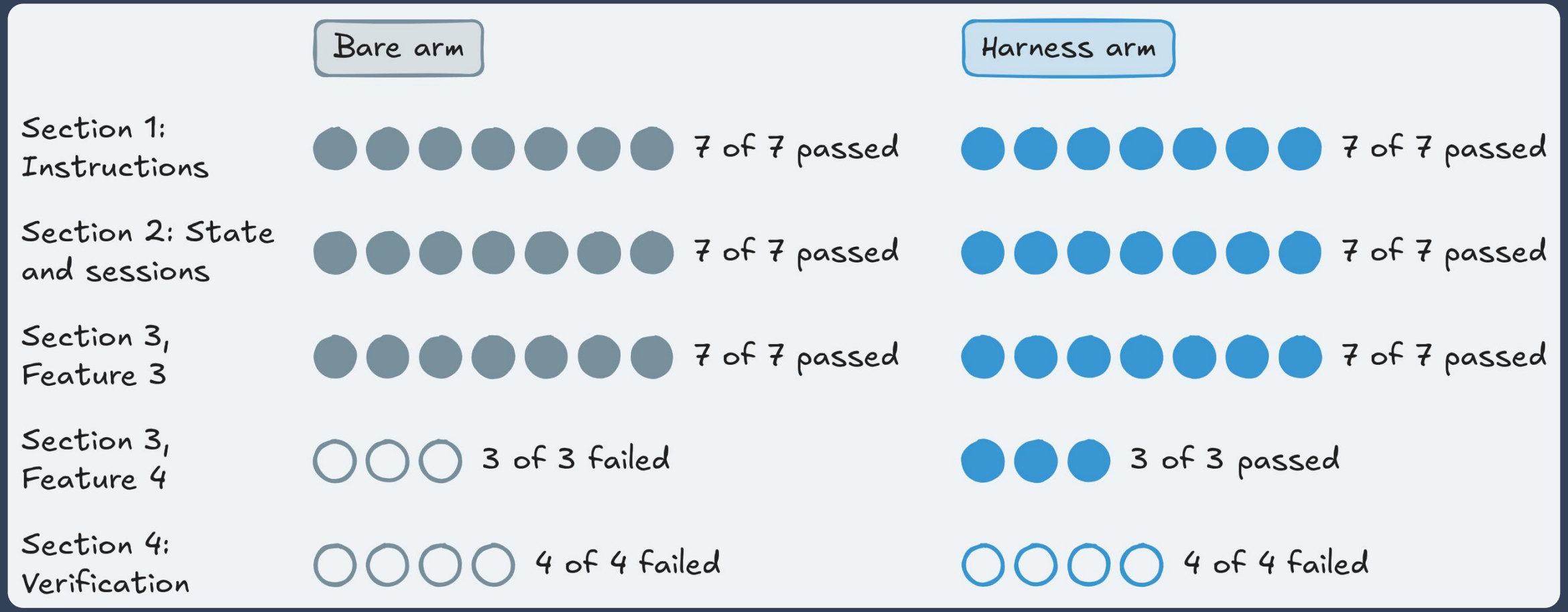
Final A/B: reading it live

The final has no primary measure: read each arm's counts.

- **hidden:** does the feature work?
- **mutants:** would its tests notice a bug?
- **claim:** did the agent say done?
- **Screen 2:** other-org or duplicate cards?
- **Screen 5:** the first wrong turn

All results: four sections, one picture

One row per section and ticket, one dot per run, bare and harness.

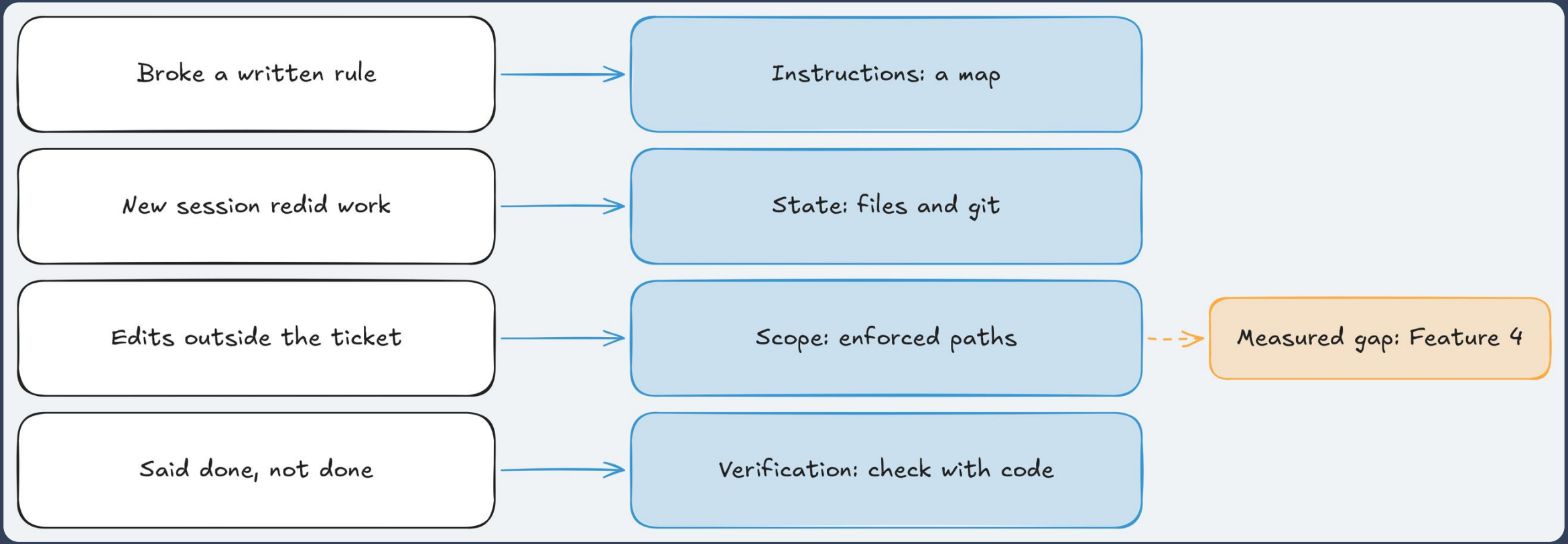


Key idea: One gap, on scope, partly by construction; elsewhere no measurable difference.

{ik} INTERVIEW KICKSTART

Which layer when: match the layer to the failure

Each layer is a bet on one thing the model cannot do on its own.



Key idea: Add the layer that matches a failure you have seen, then measure it.

{ik} | INTERVIEW KICKSTART

Use it in your repo: the kit and four skills

`uv run shipharness init --target ~/my-repo` copies the kit; `init --check` lists what to fill.

- **harness-setup:** assess a repo, fill gaps
- **assured-tests:** keep tests passing five filters
- **house-rules:** convention to rule, check, test
- **skill-template:** start your own skill

Assignments: one per layer

Starter, offline self-check, one live step, full solution.

1. **Assignment 1:** Instructions: a long AGENTS.md to map, docs, skill, checker
   `section-1-instructions/assignment/`
2. **Assignment 2:** State and sessions: three forced stops, facts from git
   `section-2-state-lifecycle/assignment/`
3. **Assignment 3:** Scope and sandbox: diff parser, scope policy, evidence-gated passes
   `section-3-scope-sandbox/assignment/`
4. **Assignment 4:** Verification: House rule 8: Deleted issues stay hidden; Filter 6: No duplicate tests
   `section-4-verification/assignment/`

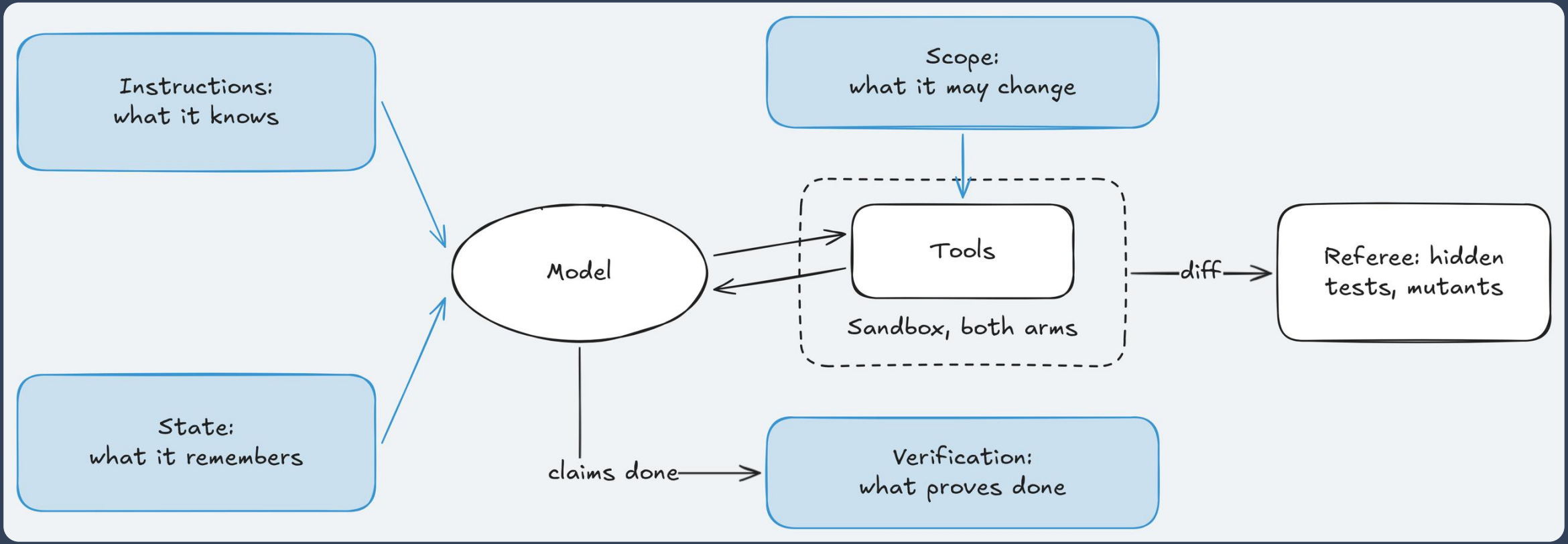
Final project: one live A/B, read as one dot

Repeat today's final on your machine.

1. Run `make final FEATURES=bulk-close`
2. Remove one layer: `LAYERS=-scope`
3. Read counts, never one verdict
4. Draw your dots: `uv run shipharness report`

The harness in one picture

Four layers around an unchanged model, one sandbox, one referee outside.

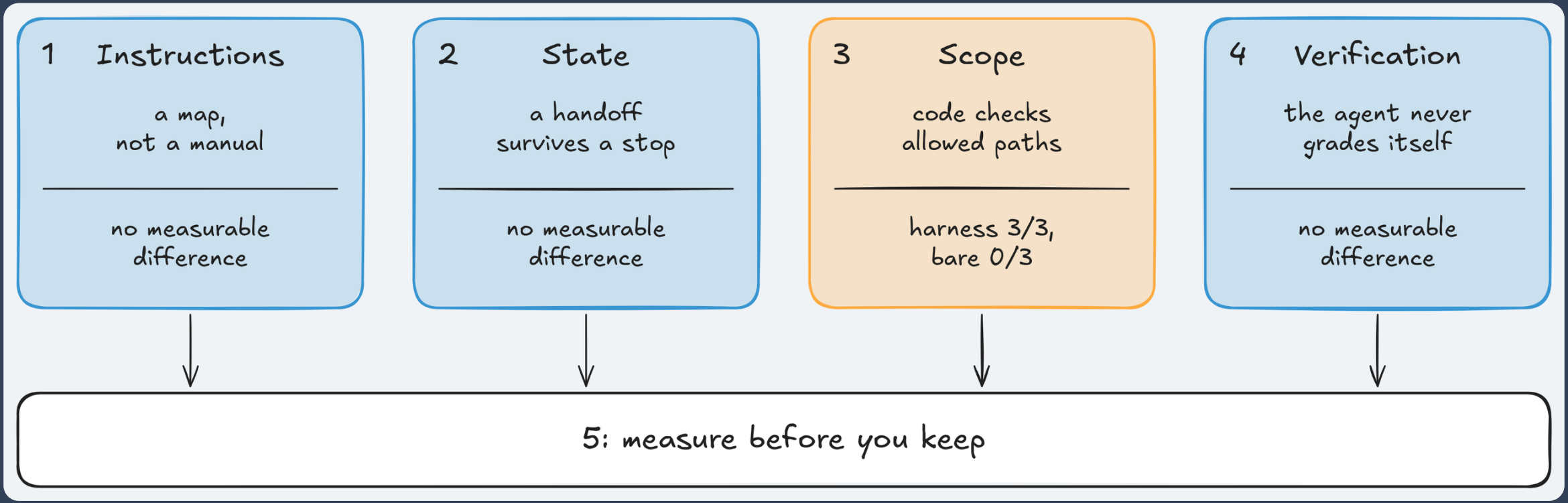


Key idea: Each layer changes what the agent sees, keeps, may touch, or must prove.

{ik} INTERVIEW KICKSTART

Module 1 Summary: Key takeaways

Four layers, four measurements, one rule.



Key idea: Every layer is a bet on what the model cannot do alone; measure it before you keep it.

Glimpse of Module 2: Loop Engineering for Coding Agents

Module 1 built the harness around one run; Module 2 builds the loop around many.

- ⚡ **Background and event-driven runs:** agents that start on events
- ↗ **Hill climbing:** change one thing, keep it if the score rises
- Defenses against agents that actively game checks
- ↻ **pass^k reliability:** does it pass every one of k tries?
- ▣ Detailed sandbox design

{ik} INTERVIEW KICKSTART

References (1 of 2): sources in the order cited

Every source cited today, numbered in order of first use, with its link.

Source	Title, author, date
Source 1	Harness engineering: leveraging Codex in an agent-first world, Ryan Lopopolo, OpenAI, 2026-02-11
Source 2	Effective harnesses for long-running agents, Justin Young, Anthropic, 2025-11-26
Source 3	Harness design for long-running application development, Prithvi Rajasekaran, Anthropic, 2026-03-24
Source 4	Building a C compiler with a team of parallel Claudes, Nicholas Carlini, Anthropic, 2026-02-05
Source 5	Billing (LangSmith docs), LangChain, living page
Source 6	Agent Skills Specification, Agent Skills project, undated
Source 7	Extend Claude with skills, Anthropic, Claude Code docs, living page
Source 8	Build skills, OpenAI, Codex docs, living page
Source 9	Agent Skills (Cursor docs), Anysphere, Cursor docs, living page

References (2 of 2): the remaining sources

The rest of today's sources, with links.

Source	Title, author, date
Source 10	Using PLANS.md for multi-hour problem solving, Aaron Friel, OpenAI Cookbook, 2025-10-07
Source 11	None network driver (Docker docs), Docker, living page
Source 12	SpecBench: Measuring Reward Hacking in Long-Horizon Coding Agents, Zhao, Srikanth, Wu, Jiang, Weco AI (repository), 2026-05-20
Source 13	Automated Unit Test Improvement using Large Language Models at Meta, Alshahwan et al., Meta, FSE 2024
Source 14	Mutation-Guided LLM-based Test Generation at Meta, Foster et al., Meta, 2025
Source 15	Autonomous Coding Agent Demo (README), Anthropic, 2025-11-26
Source 16	What are skills?, Agent Skills project, undated
Source 17	How to add skills support to your agent, Agent Skills project, undated

No Question Is Too Small Every Question Deserves an Answer!!

Ask about anything:

- Concepts from today's modules
- How this applies to your work
- Implementation details
- Clarifications on what we covered

Quick Request

- Please do not mind if a question gets skipped accidentally
- Repost politely in chat if needed
- We will try our best to cover as many questions as possible

Remember: If you're wondering about it, others probably are too.





{ik} | INTERVIEW KICKSTART